

Face Recognition Attendance System - Database Design

هذا الملف يلخص الجداول المطلوبة لربط الموقع بقاعدة بيانات حقيقية، بحيث يخدم تسجيل الدخول، المواد والشعب، جلسات الحضور، وسجلات الحضور الناتجة من المودل.

الفكرة العامة

- النظام يحتاج قاعدة بيانات بسيطة، لأنها تخدم نظام حضور بالتعرّف على الوجه وليس نظام جامعة كامل.
 - الأدمن يشاهد التقارير العامة، والإنستركتور يشاهد جلساته ومواده فقط.
 - المودل يسجل الطالب ك Present عندما يتعرف على وجهه، والطلاب غير المتعرف عليهم يُحسبون Absent في التقرير.
 - لا نحتاج جدول Reports منفصل، لأن التقرير يُولّد من sessions + attendance + students + courses + sections.
- الجداول المطلوبة باختصار

الجدول	الوظيفة
users	تخزين حسابات الأدمن والإنستركتور وصلاحياتهم.
students	تخزين بيانات الطلاب وصورة/بيانات الوجه لاحقًا.
courses	تخزين أسماء المواد.
sections	تخزين الشعب وربطها بالمادة والإنستركتور والوقت.
section_students	ربط الطلاب بالشعب.
sessions	تخزين جلسات الحضور التي يبدأها الإنستركتور.
attendance	تخزين الطلاب الذين تم التعرف عليهم كحاضرين.

users (1) جدول

هذا الجدول لتسجيل دخول الأدمن والإنستركتور.

```
CREATE TABLE users (
  id INT PRIMARY KEY AUTO_INCREMENT,
  username VARCHAR(50) UNIQUE NOT NULL,
  password VARCHAR(255) NOT NULL,
  role ENUM('admin', 'instructor') NOT NULL
);
```

مثال بيانات: 123 / admin بدور admin، و 123 / instructor بدور instructor. لاحقًا الأفضل تخزين كلمة المرور بشكل مشفر.

students (2) جدول

هذا الجدول للطلاب الذين سيتعرف عليهم النظام.

```
CREATE TABLE students (
  id INT PRIMARY KEY AUTO_INCREMENT,
  student_id VARCHAR(20) UNIQUE NOT NULL,
  name VARCHAR(100) NOT NULL,
  department VARCHAR(50),
  face_image VARCHAR(255)
);
```

الحقل face_image يمكن أن يحتوي مسار صورة الطالب مثل static/faces/12001.jpg أو يمكن استبداله لاحقًا بـ face encoding خاص للـ face.

courses (3) جدول

هذا الجدول يمثل أسماء المواد فقط.

```
CREATE TABLE courses (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  course_name VARCHAR(100) NOT NULL  
);
```

sections (4 جدول)

هذا جدول مهم لأنه يربط المادة بالشعبة والإنستركتور ووقت المحاضرة.

```
CREATE TABLE sections (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  course_id INT NOT NULL,  
  section_name VARCHAR(10) NOT NULL,  
  instructor_id INT NOT NULL,  
  day VARCHAR(20),  
  start_time TIME,  
  end_time TIME,  
  
  FOREIGN KEY (course_id) REFERENCES courses(id),  
  FOREIGN KEY (instructor_id) REFERENCES users(id)  
);
```

مثال: Web Development - Section A - Instructor X - Wednesday 08:00 to 12:00

section_students (5 جدول)

هذا الجدول يحدد الطلاب الموجودين في كل شعبة.

```
CREATE TABLE section_students (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  section_id INT NOT NULL,  
  student_id INT NOT NULL,  
  
  FOREIGN KEY (section_id) REFERENCES sections(id),  
  FOREIGN KEY (student_id) REFERENCES students(id)  
);
```

sessions (6 جدول)

كل مرة يضغط الإنستركتور Start Session يتم إنشاء سجل جديد هنا.

```
CREATE TABLE sessions (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  section_id INT NOT NULL,  
  started_at DATETIME NOT NULL,  
  ended_at DATETIME,  
  status ENUM('active', 'closed') DEFAULT 'active',  
  
  FOREIGN KEY (section_id) REFERENCES sections(id)  
);
```

عند إنهاء الجلسة يصبح status = closed ويتم تعبئة ended_at بالوقت الحالي.

attendance (7 جدول)

هذا أهم جدول للمودل. عندما يتعرف المودل على طالب، يضيف سجله هنا ك Present.

```
CREATE TABLE attendance (  
  id INT PRIMARY KEY AUTO_INCREMENT,
```

```

    session_id INT NOT NULL,
    student_id INT NOT NULL,
    status ENUM('Present', 'Absent') DEFAULT 'Present',
    detected_at DATETIME,

    FOREIGN KEY (session_id) REFERENCES sessions(id),
    FOREIGN KEY (student_id) REFERENCES students(id),

    UNIQUE (session_id, student_id)
);

```

وجود UNIQUE(session_id, student_id) مهم حتى لا يتم تسجيل نفس الطالب أكثر من مرة في نفس الجلسة إذا تعرفت عليه الكاميرا أكثر من مرة.

العلاقات بين الجداول

```

users
-> instructor teaches sections

courses
-> has sections

sections
-> has students through section_students
-> has sessions

sessions
-> has attendance records

attendance
-> student is Present in a specific session

```

كيف يتغير الكود الحالي بعد الداتا بيز؟

بدل القوائم المؤقتة داخل app.py مثل:

```

students = [...]
courses = [...]
sessions = []
attendance = []

```

سيتم جلب البيانات من قاعدة البيانات، مثل:

```

students = Student.query.all()
courses = Course.query.all()
sessions = Session.query.all()
attendance = Attendance.query.all()

```

أو باستخدام MySQL مباشر:

```

cursor.execute("SELECT * FROM students")
students = cursor.fetchall()

```

منطق الغياب والحضور

- عند بداية الجلسة، كل طلاب الشعبة يعتبرون Absent افتراضياً.
 - عندما يتعرف المودل على وجه طالب، يتم إدخاله في attendance ك Present.
 - في التقرير، الغياب يحسب هكذا: طلاب الشعبة - الطلاب المسجلون في attendance = Absent.
 - الأفضل حالياً تخزين الحضور فقط، وحساب الغياب عند عرض التقرير.
- أقل قاعدة بيانات ضرورية للمشروع

users
students
courses
sections
section_students
sessions
attendance

الخلاصة

هذه الجداول كافية لتشغيل الموقع بشكل حقيقي: تسجيل دخول، صلاحيات، مواد وشعب، جلسات حضور، تسجيل حضور من المودل، وتقارير مع PDF. لا حاجة لجدول Reports منفصل لأن التقارير تُستنتج من الجداول السابقة.

Prepared for Face Recognition Attendance System